Database Querying in SQL

Refining Your Query: *You need to get some data from the "film" table and decide to use the query SELECT * FROM film.*

You realize that only the "film_id" and "title" columns are needed. Write a new query that selects only those 2 columns.

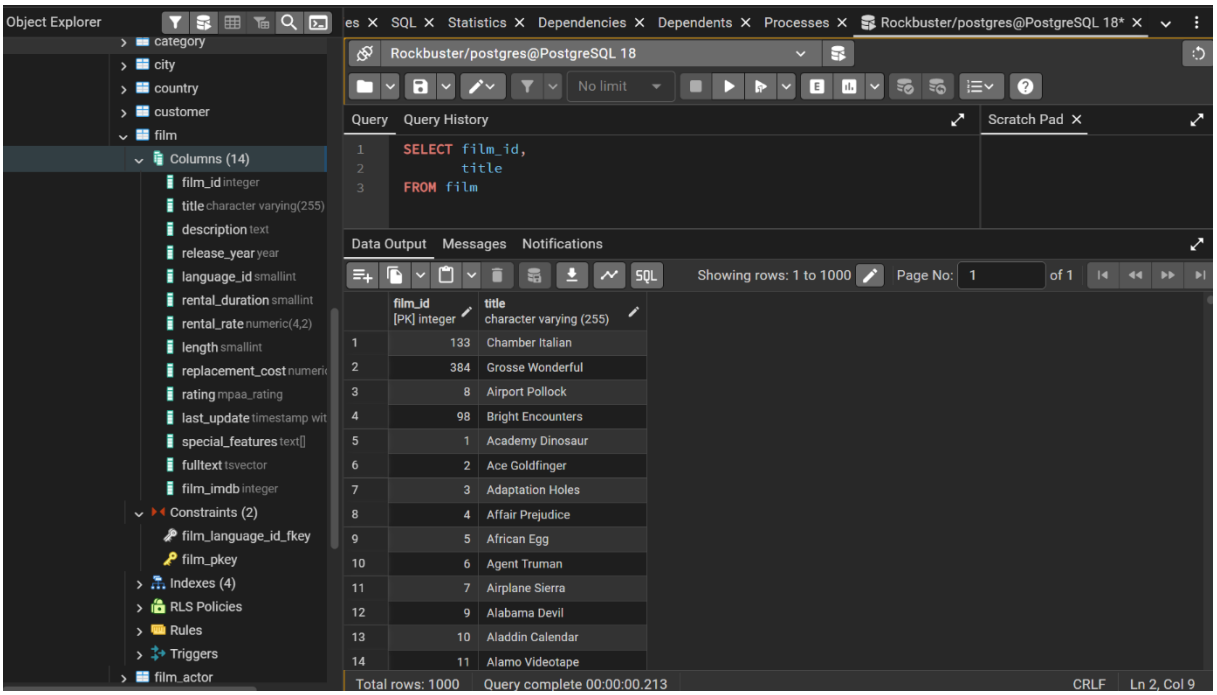
Compare the cost of the original query and the revised query, and write a few sentences explaining the comparison. Can you suggest any ways to optimize this query?

Revised Query:

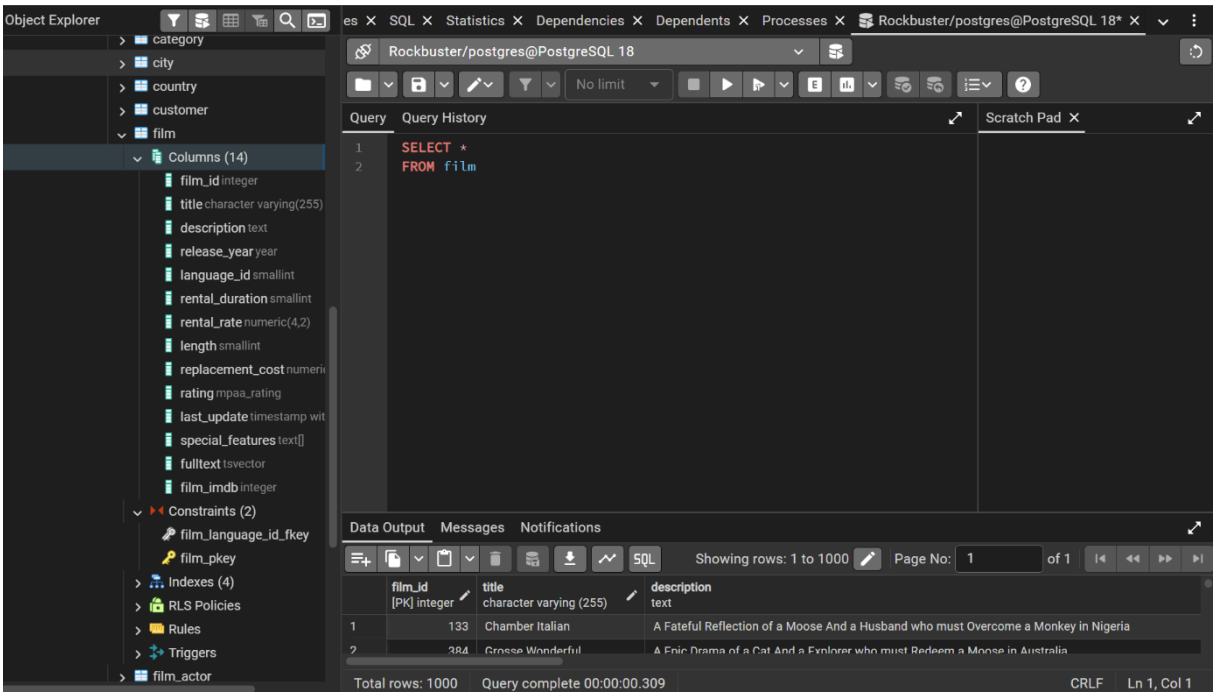
```
SELECT film_id,
                title
 FROM film
```

When comparing the original query (SELECT * FROM film) and the revised query (SELECT film_id, title FROM film), both query plans show a sequential scan on the film table with a similar estimated cost. This is because the query still needs to scan all rows in the table to return the results, and no filtering condition is applied.

However, the width of the rows returned by the revised query is smaller because fewer columns are selected. This means less data is read from disk and transferred in memory, which can improve performance, especially when working with large tables or when queries are run frequently.



In contrast, SELECT * retrieves unnecessary columns, increasing data transfer and memory usage without adding value to the result.

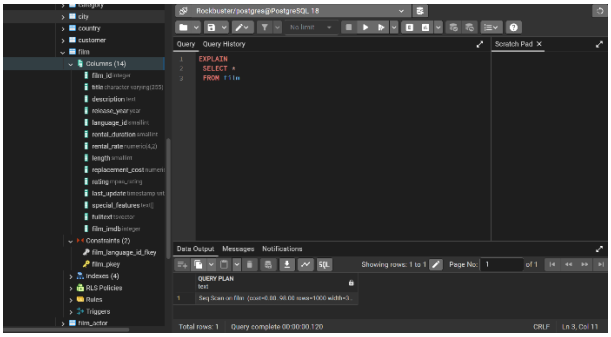


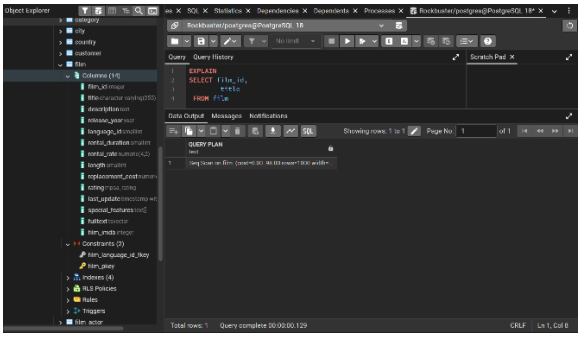
While this query is already more efficient than SELECT *, further optimization could include:

- Creating an index on film_id if this column is frequently used in joins or filters.
- Avoiding SELECT * as a best practice to improve performance and readability.

Overall, selecting only the required columns is a good optimization step and improves query efficiency, even when the execution plan appears similar.

Why the cost remains the same/similar: The EXPLAIN output shows a similar estimated cost for both queries because PostgreSQL performs a sequential scan in both cases and must read all rows in the film table. Since no filtering conditions are applied, the optimizer estimates the same total cost for scanning the table. However, the revised query has a smaller row width because it selects only the required columns, which reduces the amount of data processed. While the estimated cost remains similar, selecting fewer columns improves efficiency and is considered best practice, particularly for larger datasets.

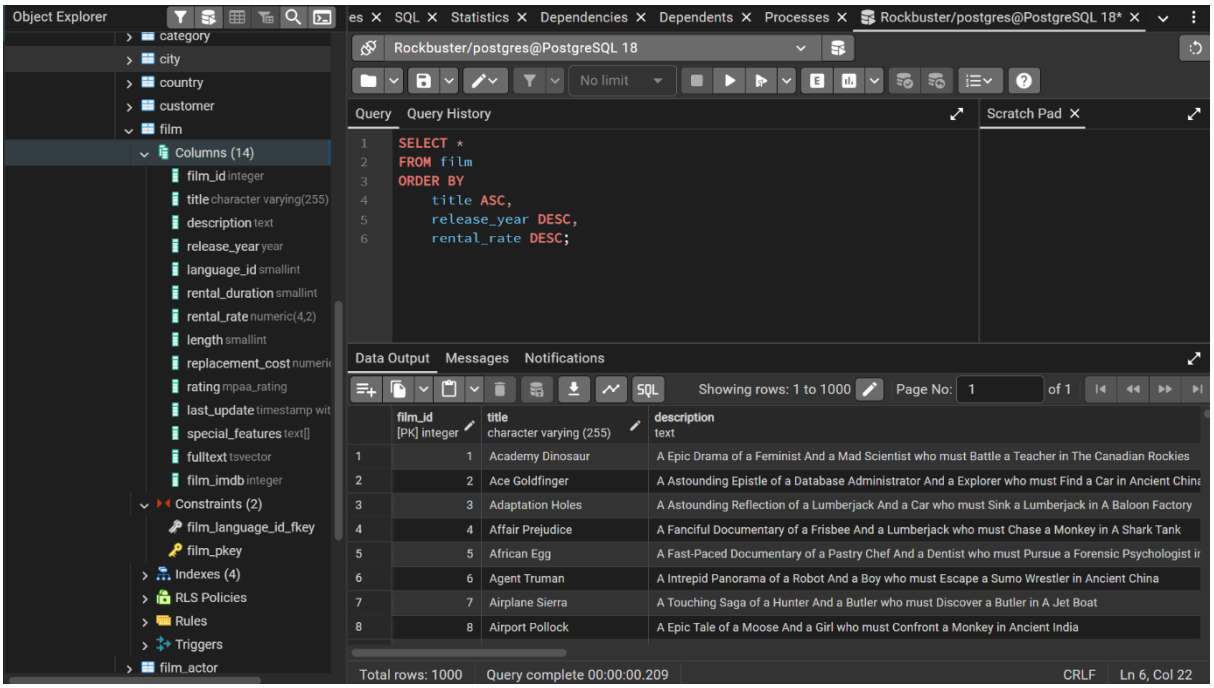




Ordering the Data: *In the pgAdmin Query Tool, run a query that selects every film from the "film" table, with the movies sorted by title from A to Z, then by most recent release year, and then by highest to lowest rental rate.*

Extract the data output of your query into a CSV file for the film collection department to analyze in Excel. To do this, click the button "Save results to file":

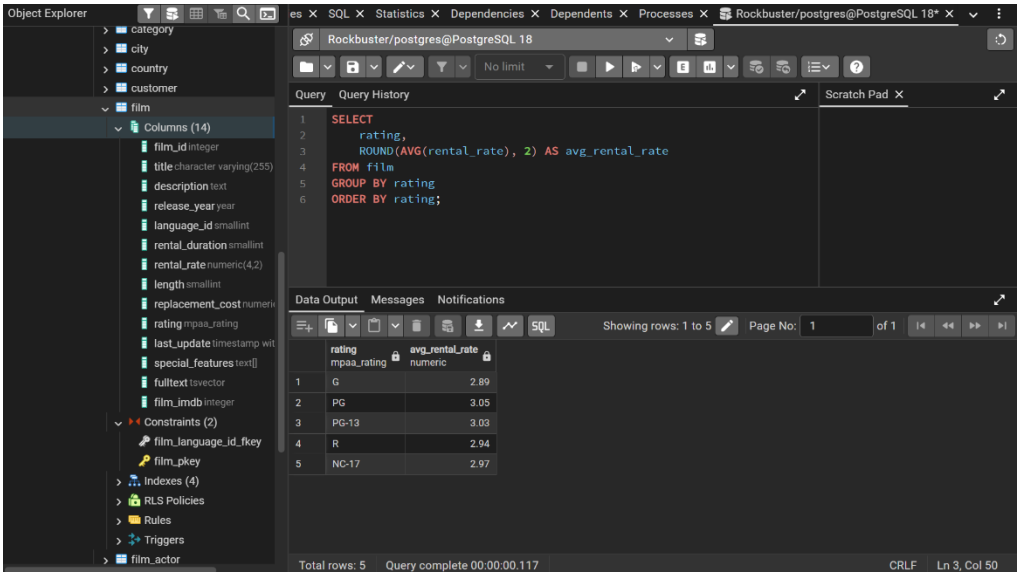
```
SELECT *
FROM film
ORDER BY
  title ASC,
  release_year DESC,
  rental_rate DESC;
```



Grouping Data: *The strategy department has asked you the questions below. Write a SQL query to retrieve the correct answers, then extract your results as a CSV file.*

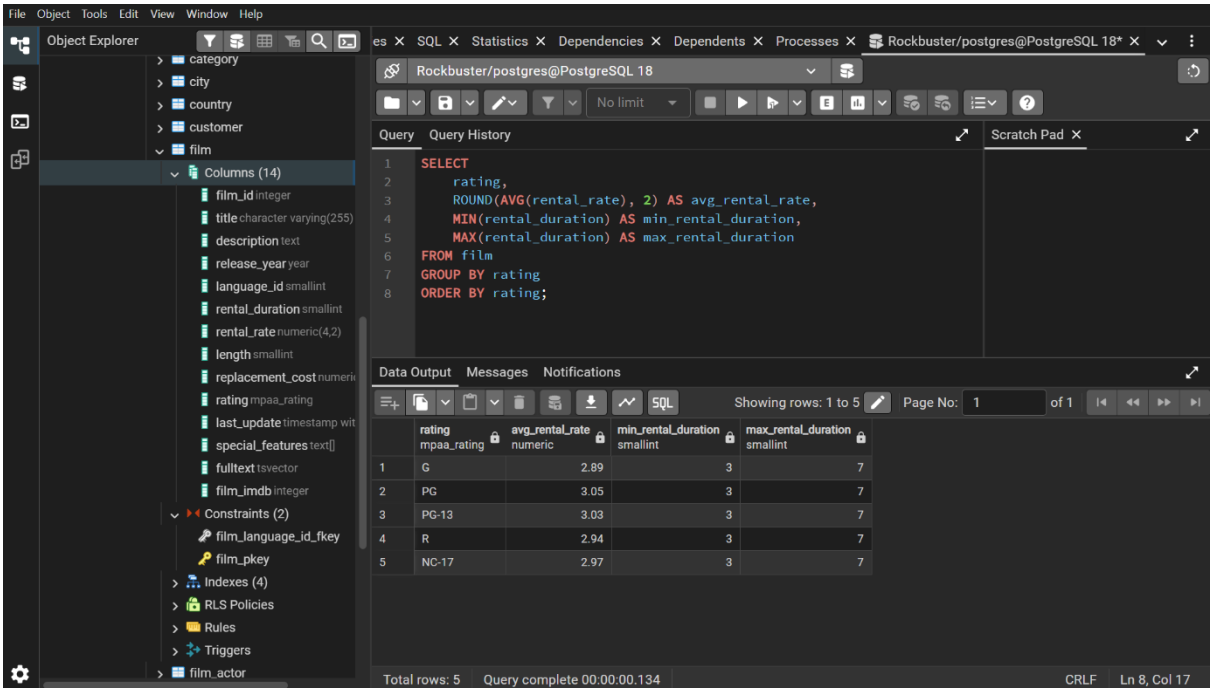
- *What is the average rental rate for each rating category?*

Average Rental Rate =



What are the minimum and maximum rental durations for each rating category?

Min and Max rental durations for each rating category =



Database Migration: *Your team has decided to use an external tool to collect data on user behavior in the new Rockbuster Android app. Data collected from this new source will need to be loaded into the data warehouse before you can analyze it.*

Can you outline the procedure for migrating the data and who will be responsible for it?

To migrate data collected from the external user behavior tracking tool into the Rockbuster data warehouse, the process would typically follow several steps. First, the raw data would be extracted from the external tool, either through scheduled exports or API connections. This step is usually handled by data engineers or the technical team responsible for managing data pipelines.

Next, the extracted data would be transformed to match the structure and standards of the existing data warehouse. This includes cleaning the data, validating formats, handling missing values, and mapping fields to the appropriate tables. Data engineers are primarily responsible for this transformation step, often using ETL (Extract, Transform, Load) tools.

Finally, the transformed data would be loaded into the data warehouse. Once loaded, the data is tested and validated to ensure completeness and accuracy. At this stage, data analysts may assist by performing validation queries and confirming that the data is ready for analysis.

What problems do you foresee if you start analyzing the data before it's been loaded into the data warehouse?

Analyzing the data before it has been properly loaded into the data warehouse can lead to several problems. The data may be incomplete, inconsistent, or unvalidated, which can result in inaccurate insights and unreliable conclusions. Without proper transformation and standardization, metrics may not align with existing definitions used across the organization.

Additionally, analyzing data outside the warehouse can lead to version control issues, where different teams work with different copies of the data. This increases the risk of conflicting results and reduces trust in the analysis. The data warehouse serves as a single source of truth, so performing analysis before the data is fully integrated can undermine data quality and decision-making.